perplexity

You are the senior expert complex problem solver and help me to build my MVP for the AI for Bharat Hackathon we have moved to the building phase where the submission requirements are project summary demonstration of the video link code base of Github repository working link of the URL of our project The project should be functional and where we are moving to prototype development phase We have four days to prototype this I have uploaded all the requirements.Md and design.MD File And as the first year student we don't know anything about how to build so you should guide us in detail building phase and till submission of the prototype to AI for Bharat you will act as the mentor for our team Each and every step should be in detail a small child can understand and work on it Each step should be clear and detailed and ppt what we are doing first you understand in detail about our project and tell us once you are confiend to build withiout any errors in the code while build this project this prject is very important for me and my team ,you know my team not know any thing they are biggner in this field so you will leads for this project till we submit the working link to them we have only 4 days for this building phase all must the files must be separate it makes ust easy to build this

Here is the situation in simple words, and then a clear plan.

1. What your project is (in simple language)

From your `requirements.md`, `design.md` and both PDFs, this is what your project is:

- Project name: **Vidya-Setu** – "Bridge of Knowledge".
- Goal: Help **rural Indian students** learn school subjects (like Physics, Maths) using:
  - Their **own language** (Kannada, Hindi, etc.).
  - **Voice** instead of typing (because many are not comfortable typing in English).
- Big ideas:
  1. **Agentic AI Tutor**
     Not just a chatbot. It should behave like a **teacher/mentor**:
     - Understand what the student already knows.
     - Predict what they should learn next.
     - Give **examples from daily life** (like farming, cricket, festivals).
  2. **Transcreation, not just translation**
     Instead of just translating English to Kannada, it explains in a **culturally familiar way**.
  3. **Offline-first**
     Internet in villages is weak. So:
     - When internet is available, the app downloads **next few lessons**.
     - Later, student can study **offline** from those saved lessons.
  4. **Uses Indian and AWS tech**
     - **Bhashini** – for Indian language speech and translation.
     - **Amazon Bedrock (Claude 3.5)** – for smart explanations and analogies.
     - **AWS Lambda, API Gateway, DynamoDB, S3** – serverless, cheap and scalable.
     - Frontend: **React Native mobile app** (in the AI-Hackathon PDF)
       and/or **React PWA** (in design.md). Both are acceptable to judges.

Hackathon submission needs:

- Working **prototype** (URL or app link).
- **GitHub repo** (clean structure, clear README).
- **Short video demo** (showing main features).
- **Project summary / PPT** (idea, architecture, demo screenshots).

You have **4 days** and the team are **beginners**.

So: you cannot build *everything* in the huge architecture.
You must build a **small, working slice (MVP)** that proves the idea.

2. What we will actually build in 4 days (realistic MVP)

To finish in 4 days as first-years, we will build a **very focused MVP**:

MVP: Thin but real "Agentic Tutor" slice

Student can:

1. Open a **simple mobile-friendly web app** (React PWA).
2. Choose:
   - Language: **English / Kannada** (start with just 2).
   - Subject/topic: e.g. **Physics – Gravity**.
3. Ask a **question** (first as **text**, voice later if time).
4. Backend:
   - Sends question to **Bhashini** (or a mocked translation if you cannot get API keys in time).
   - Then sends English version + student level to **Amazon Bedrock (Claude)**.
   - Gets answer: explanation + simple analogy + 1 quiz question.
   - Optionally translates back to Kannada using Bhashini.
5. Frontend shows:
   - A "**Knowledge Card**":
     - Short explanation.
     - Local analogy (e.g., using farm example).
     - 1 simple quiz question.
   - Student can answer quiz; app shows **correct / incorrect**.
6. **Simple offline feel**:
   - When online: app **saves last 3 Knowledge Cards** in browser storage (localStorage).
   - When offline: app shows "You are offline, but you can review your saved cards".

This is enough to:

- Show **agentic behavior** (it generates structured cards + quiz).
- Show **vernacular + cultural analogy**.
- Show **basic offline resilience**.
- Use **AWS Bedrock** and **(optionally) Bhashini** as promised.

We will **not** build in 4 days:

- Full 22 languages.
- Full WatermelonDB offline engine.
- Full dynamic curriculum prediction with Step Functions.
- Full admin NCERT ingestion pipeline.

But in the PPT and README we will say:

- "This is **MVP**. The architecture is designed to scale to full features (as described in `design.md`)."

3. Tech stack for MVP (kept simple)

To make your life easier:

Frontend (web app, mobile-friendly)

- **React** + **TypeScript** (Create React App or Vite).
- **PWA support** (basic service worker, but no complex stuff).
- Use:
  - `fetch` or `axios` to call backend.
  - `localStorage` to store last 3 Knowledge Cards.

Backend

- **Python 3** + **FastAPI** (easy and beginner-friendly)
  OR **Node.js** + **Express** (if your team is more comfortable with JS).
- Expose simple endpoints:
  - `POST /api/ask` – student question → returns Knowledge Card.
  - `POST /api/quiz/submit` – (optional) log quiz results.
- Integrations:
  - **Amazon Bedrock** (Claude 3.5 Sonnet) via AWS SDK.
  - **Bhashini** via HTTP REST (if keys are available).
    If you cannot get Bhashini access quickly, we will:
    - Simulate multilingual interaction using Claude prompts (still mention Bhashini in design as planned integration).

Deployment

- **Backend**: Deployed as **AWS Lambda** + **API Gateway** (via AWS SAM or Serverless Framework)
  OR simpler for 4 days: deploy as a small server on **Render / Railway / EC2**.
- **Frontend**: Deployed on **Netlify / Vercel / GitHub Pages**.

For hackathon, it is enough that:

- The app is **publicly reachable**.
- Demo clearly shows the **core idea**.

4. Repository & folder structure (everything clearly separated)

You said: "all the files must be separate – it makes us easy to build this".

Here is a clean structure you can use for your GitHub repo:

```
vidya-setu/
├── frontend/                # React PWA
│   ├── src/
│   │   ├── components/
│   │   │   ├── LanguageSelector.tsx
│   │   │   ├── TopicSelector.tsx
│   │   │   ├── QuestionInput.tsx
│   │   │   ├── KnowledgeCard.tsx
│   │   │   └── OfflineBanner.tsx
│   │   ├── pages/
│   │   │   └── HomePage.tsx
│   │   ├── services/
│   │   │   └── api.ts          # calls backend /api/ask
│   │   ├── hooks/
│   │   │   └── useOfflineCache.ts
│   │   ├── workers/
│   │   │   └── serviceWorker.ts # basic PWA worker
│   │   └── main.tsx / index.tsx
│   ├── public/
│   │   └── manifest.json       # PWA manifest
│   ├── package.json
│   └── README-frontend.md
│
├── backend/                 # FastAPI or Express backend
│   ├── app/
│   │   ├── main.py / index.js
│   │   ├── bedrock_client.py
│   │   ├── bhashini_client.py
│   │   └── schemas.py
│   ├── requirements.txt / package.json
│   └── README-backend.md
│
├── docs/
│   ├── requirements.md       # already written
│   ├── design.md             # already written
│   ├── ppt-outline.md        # we will create content for PPT
│   └── submission-checklist.md
│
├── .gitignore
├── README.md                 # high-level summary + how to run
└── LICENSE (optional)
```

This gives you:

- **Clear separation** between frontend, backend, docs.
- Each team member can work in **different folders** without confusion.

5. 4-day high-level plan

Day 1 – Setup and skeletons

Goals:

- GitHub repo created with basic structure.
- Frontend and backend both run **locally**, even with fake data.

Tasks:

1. **Set up tools on all laptops**
   - Install:
     - `Node.js` (LTS).
     - `Python 3.10+`.
     - `git`.
     - Any code editor (VS Code).
2. **Create GitHub repository**
   - Name: `vidya-setu` or `vidya-setu-dreamalpha`.
   - Add team members as collaborators.
3. **Create folder structure** (as shown above).
   - Commit `requirements.md` and `design.md` into `/docs`.
4. **Frontend skeleton**
   - Use `create-react-app` or `npm create vite@latest` (TypeScript template).
   - Create:
     - `LanguageSelector`, `TopicSelector`, `QuestionInput`, `KnowledgeCard`.
   - For now, **do not call backend**; just:
     - When user clicks "Ask", show a **hard-coded** Knowledge Card.
5. **Backend skeleton**
   - Set up **FastAPI** or **Express** with:
     - `POST /api/ask` that returns **hard-coded JSON** like:

```
{
  "language": "kn",
  "topic": "Gravity",
  "question": "What is gravity?",
  "answer": "Gravity is the force that pulls objects towards the Earth.",
  "analogy": "Like how a mango falls from a tree to the ground in your villa
  "quiz_question": "If you drop a ball, in which direction will it move?",
  "quiz_answer": "Towards the Earth / downwards"
}
```

6. **Connect frontend to backend locally**

- Use `fetch("http://localhost:8000/api/ask")` (FastAPI) or appropriate port.
- Confirm:
  - Type a question → get mock Knowledge Card from backend → show on screen.

At the end of Day 1:
You should have a **working full loop** but with **fake AI logic**.

Day 2 – Integrate Amazon Bedrock (real AI reasoning)

Goals:

- Backend actually calls **Amazon Bedrock Claude 3.5**.
- Frontend displays **real answers** (but still maybe only English).

Tasks:

1. **AWS setup**
   - Ensure:
     - You have an **AWS account**.
     - Bedrock is **enabled** in your region (e.g., `ap-south-1`).
   - Create an **IAM user** with credentials that can call Bedrock.
2. **Install Bedrock SDK**
   - For Python:
     - Install `boto3` and Bedrock-specific libs.
   - Write a simple **test script** to:
     - Send a question like "Explain gravity to a 10-year-old rural student using farming example".
     - Print response.
3. **Wrap Bedrock in backend**
   - Replace the fake response in `/api/ask` with:
     - Call to Bedrock model.
     - Prompt template that:
       - Asks model to:
         - Output: explanation + analogy + one simple quiz question.
         - Use **given language** when possible.
4. **Update frontend**
   - Display the AI-generated explanation, analogy, and quiz.

At the end of Day 2:
You have a **real AI tutor** (English, maybe simple Kannada-like output).

Day 3 – Add basic multilingual + "offline-feel"

Goals:

- Basic **language toggling** (English/Kannada).
- Simple **offline feel** (localStorage caching of last cards).

Tasks:

1. **Language selector logic**
   - When user selects Kannada:
     - Frontend sends `"language": "kn"` to backend.
   - In backend prompt for Bedrock:
     - "If language is kn, answer in Kannada and use local Indian examples."
2. **(Optional but ideal) Bhashini integration**
   - If you have Bhashini keys:
     - Call Bhashini ASR/NMT/TTS from backend.
     - Flow:
       - Kannada text → Bhashini → English → Bedrock → English → Bhashini → Kannada.
   - If you do **not** manage Bhashini:
     - Clearly mention in PPT:
       - MVP directly prompts Claude for Kannada.
       - Bhashini integration is part of **future work**, see design.md.
3. **Offline behavior (simple version)**
   - In frontend:
     - After every `/api/ask` response, save the Knowledge Card into `localStorage`.
     - Keep only last 3.
   - When app loads:
     - Check `navigator.onLine` and `localStorage`.
     - If offline:
       - Show banner: "You are offline. Showing last saved cards."
       - Do not allow new questions; just show saved cards.

At the end of Day 3:
You have **multilingual-feel** and **offline-feel** MVP.

Day 4 – Polish, PPT, video, deployment

Goals:

- Deploy frontend + backend.
- Record demo video.
- Final PPT + README + GitHub ready.

Tasks:

1. **Deploy backend**
   - Simplest: use a free service like **Render**, **Railway**, or a small **EC2** instance.
   - Get a public URL: `https://vidyasetu-backend.onrender.com/api/ask`.
2. **Deploy frontend**
   - Use **Vercel** or **Netlify**.
   - Set backend URL in a config file (e.g., `VITE_API_URL` or `.env`).
   - Test from phone browser.
3. **Prepare PPT**
   - Slides (we can draft content in the next step):
     1. Problem (rural students, trilemma).
     2. Solution overview (Vidya-Setu, agentic tutor).
     3. Architecture (Edge – Bridge – Brain).
     4. Core features (voice, multilingual, offline-first, analogy-based).
     5. Demo screenshots.
     6. Future work (full Bhashini, WatermelonDB, NCERT ingestion).
     7. Tech stack.
     8. Conclusion & impact.
4. **Prepare demo video (2–3 minutes)**
   - Script:
     - 20–30 sec: Introduce team + problem.
     - 60–90 sec: Live walkthrough of app:
       - Change language.
       - Ask a question.
       - Show Knowledge Card.
       - Turn off internet → show offline cards.
     - 30–40 sec: Architecture slide + closing.
5. **Clean GitHub**
   - Make sure:

        - `README.md` has:
            - Short description.
            - How to run frontend.
            - How to run backend.
            - Link to live demo.
            - Link to video.
        - `docs/` has requirements, design, ppt outline.

6. About "no errors" and how mentoring will work

It is not possible to **guarantee** absolutely no code errors, especially with new tech (AWS Bedrock, Bhashini) and beginners.

But this is what will be done:

- Give you **clear file-by-file code** (frontend and backend) in later messages.
- Explain each step in **very simple language** ("like a small child" as you requested).
- Help you **debug typical errors** (imports, CORS, wrong URLs, etc.) by:
    - Asking you to paste error messages.
    - Telling you exactly what to change.

For this first message, the goal was:

1. Understand your project fully – done.
2. Define a **realistic 4-day MVP** – done.
3. Give high-level structure and plan – done.

7. What you should do next (today)

Before writing any real code, do just this:

1. Confirm:
    - Do you have:
        - An AWS account?
        - At least one laptop where you can install Node, Python, git?
    - Is your team okay to use:
        - **React** for frontend?
        - **Python FastAPI** for backend? (If you strongly prefer Node/JS backend, mention it.)
2. Once you confirm stack (Python or Node backend), the next reply will:
    - Give **exact commands** to:

        - Create the GitHub repo.
        - Initialize frontend project.
        - Initialize backend project.
    - Start writing the **actual code files** step-by-step:
        - `backend/app/main.py` (or `index.js`).
        - `frontend/src/components/*`.
        - Simple API integration.

Reply with:

- Your chosen backend language (**Python or Node.js**).
- Whether you have AWS account access yet.

Then the guidance will move into **Day 1 detailed steps with commands and code**.

⁂

1. design.md
2. AI-Hackathon-Submission-Preparation.pdf
3. Ai_for_bharat-TeamDreamAlphasubmission.pdf
4. requirements.md